Analysis of the Sliding Cubes Algorithm

Programmable Matter & Distributed Algorithms

Muhammad Omais Rana, Muhammad Shiraz

Master 2 Internet of Things

UFR STGI, Université de Franche-Comté

October 11, 2026

Color	Meaning	Set by
Yellow	The leader (the planner and boss)	`restoreColor()`
Grey	A cube that is idle and **not** on a target cell	`restoreColor()`
Green	A cube that is idle and **on** a target cell	`restoreColor()`
Red	A cube that is **moving right now**	`executeCalculatedMove()`

Every cube is always in one of these four states. The program is the story of grey cubes turning red, then green.

1 Start of the simulation

Every cube runs `startup()` and calls `restoreColor()`.

- The cube at $(3, 1, 0)$ has `leader="true"` in the XML, so it turns yellow.
- Cubes that already sit on a target cell turn green.
- Everything else is grey.

Only the yellow leader continues. It lists all cubes and targets, then calls `startNextRound()`. All other cubes return from `startup()` and wait for messages.

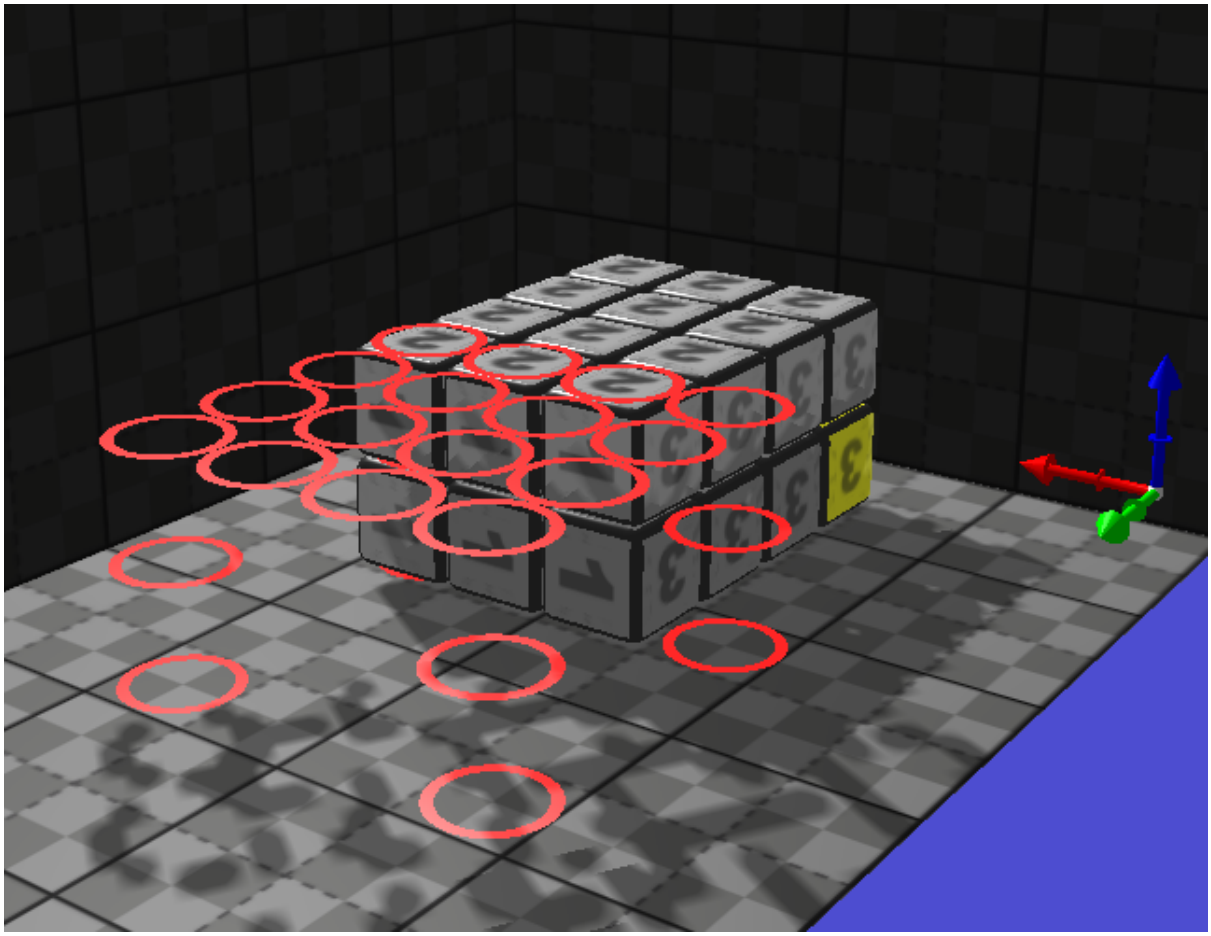


Figure 1: Leader marked.

2 The leader plans (nothing moves)

The yellow leader runs `buildPlan()`. This is pure thinking so no cube changes color, and the window may freeze briefly while A* searches.

The result is a **recipe**, an ordered list of single moves such as `(3,4,1)` → `(3,4,2)`. Planning is described in detail further down.

3 One round, color by color

Each step of the recipe is one round.

1. **All cubes idle (grey, green, yellow).** The leader takes the next step from the recipe and checks it against the real world, the source cell must hold a cube and the destination must be empty.

2. **The order is sent.** The leader sends a `MoveToken` to its neighbors, who relay it onward. All cubes are still in their idle colors.

3. **One cube recognizes its own position.** In `receiveMove()`, the cube whose position equals `token.from` decides "that's me". Every other cube relays the message and does nothing else.

4. **That cube turns RED.** `executeCalculatedMove()` runs three safety checks: no forbidden stacking, destination empty, and `canMoveTo()` allows it. If all pass, it calls `setColor(RED)` and `moveTo()`. The cube slides while red.

5. **Motion ends.** `onMotionEnd()` fires and `restoreColor()` runs.

   - If the cube landed on a target cell it becomes **green**.
   - Otherwise it goes back to **grey**.
   - If it is the leader, it returns to **yellow**.

6. **The result is reported.** The cube sends a success message back, flooded through its neighbors. The leader receives it, advances to the next step, and the loop repeats.

The pattern you should see is: **one cube turns red, lands, becomes green or grey, then the next cube turns red.** Only one cube is red at a time.

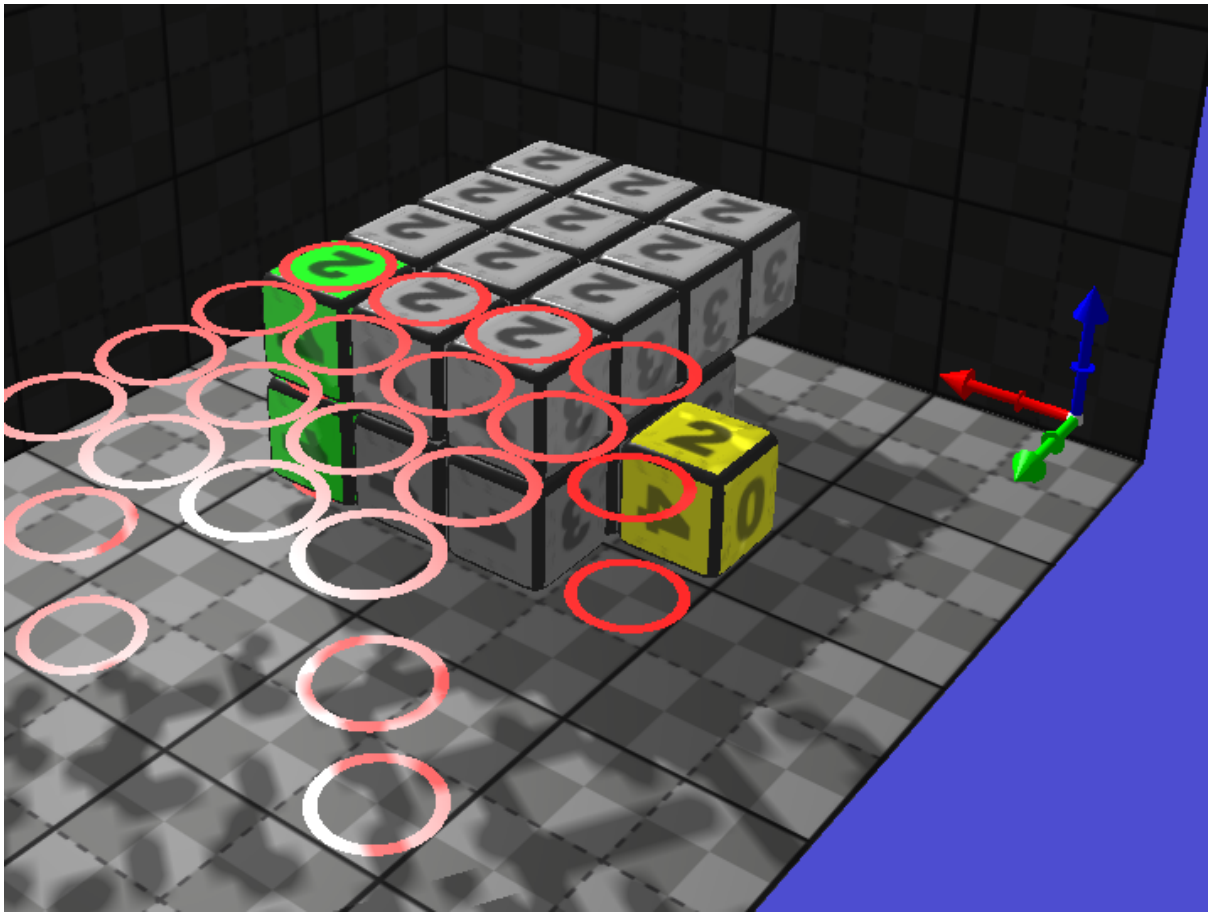


Figure 2: A round completed. Leader makes a move and the blocks already present in target cells turn green.

```
Cubes = 24 | targets = 24 | initial distance = 72
Planning from current configuration...
PLAN FOUND (weight 4): 99 individual motions, 400 expansions.

MOTION ROUND 0 | STEP 1/99 | (3,1,0) -> (2,2,0)
EXECUTING: (3,1,0) -> (2,2,0)
MOTION FINISHED: (3,1,0) -> (2,2,0)
Result OK for round 0 | total motions = 1
```

Figure 3: Console log of the round.

4 How the move and the block are selected

4.1 The block is chosen by position, not by name

The plan only says "move whatever is at position A to position B." Cubes have no assigned roles. The block that executes the move is simply **the one standing on the `from` cell**. Every cube compares its own position with `token.from`, and only the match acts.

4.2 The move is chosen by the A* planner

Step 1: Generate every legal move (`generateAbstractMoves`). For each cube and each of 48 rules (24 slides, 24 rotations), a move is kept only if:

- the destination is inside the grid and empty,
- the support and clearance cells the rule needs are in the right state (occupied for support, free for the swing path),
- the move is not in the leader's `forbiddenMoves` list, and
- after the move, all cubes still touch face-to-face as one piece.

Step 2: Score the resulting configurations. For each reachable configuration, A* computes:

$$f = g + w \times h$$

- g is the number of moves made so far.
- h is the Hungarian cost: the total distance if every cube were matched optimally to a distinct target cell. This is how "closer to the table" is measured.
- w (the weight: 4, then 8, then 16 for 24 cubes) makes the search favor "close to the goal" over "short path". The plan is found quickly but is not guaranteed to be the shortest.

Step 3: Always expand the lowest-f configuration next. Ties go to the one with larger g. This repeats until the goal configuration is reached, giving up after 150,000 expansions and retrying with the next weight.

Step 4: Read the path back. From the goal, follow the parent links back to the start. The moves on that path, in order, become the recipe.

The key subtlety is that the planner does not pick "which cube". It picks the best **sequence of configurations**, and each step of that sequence happens to involve one cube moving from one cell to another. The cube-to-target matching only exists inside the heuristic to steer the search.

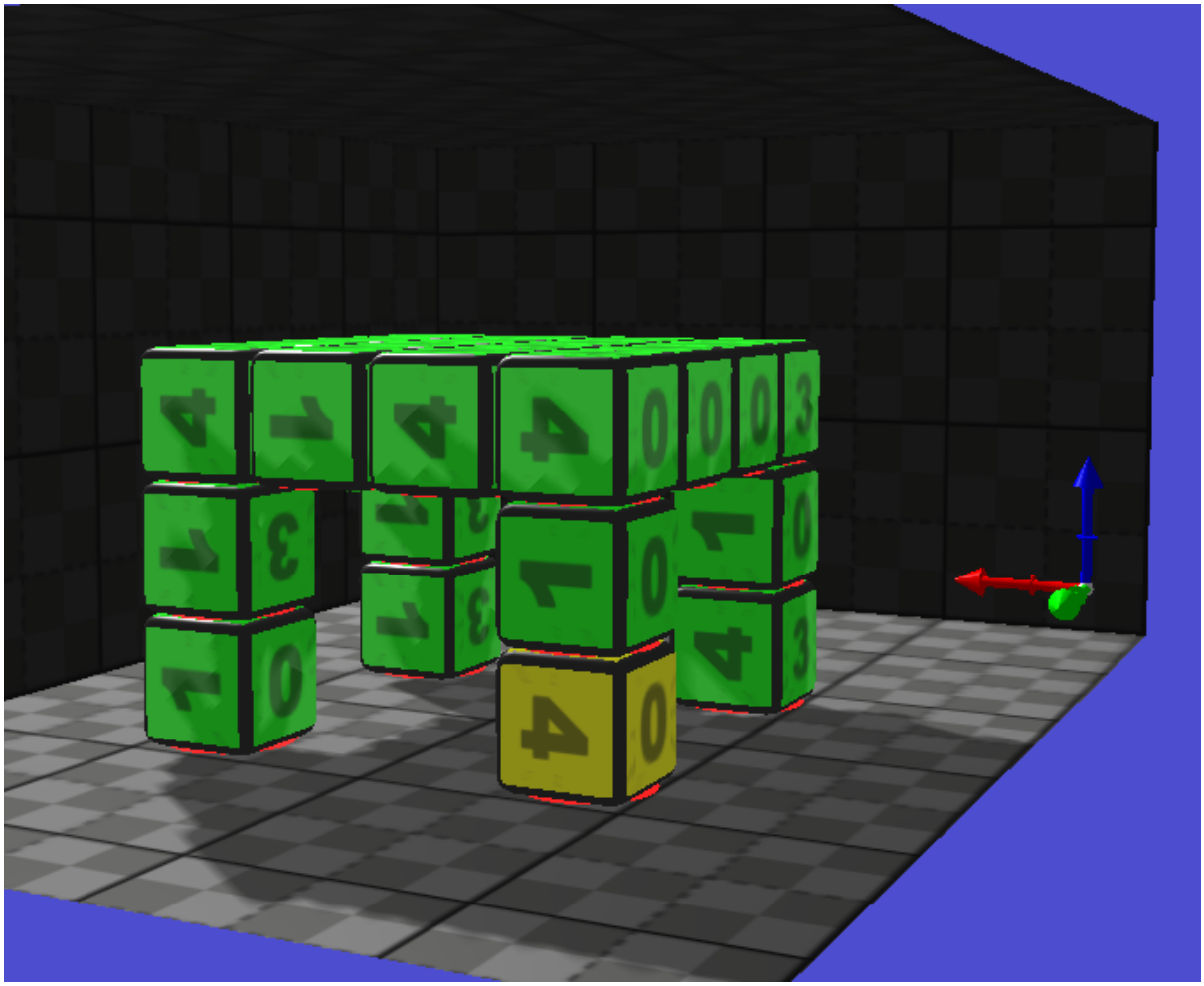


Figure 4: The state at the end of the program.

```
MOTION ROUND 98 | STEP 99/99 | (2,5,1) -> (2,4,0)
EXECUTING: (2,5,1) -> (2,4,0)
MOTION FINISHED: (2,5,1) -> (2,4,0)
Result OK for round 98 | total motions = 99
TARGET COMPLETED. Total individual motions = 99
```

Figure 5: Console log at the end of the program.

4.3 What happens when a move is rejected

If a cube turns down a move before going red (occupied destination, or `canMoveTo()` false), it sends **failure**. The leader adds that exact move to `forbiddenMoves`, discards the rest of the recipe, and re-plans from the **live** configuration. You will see no red flash for that step, because the check fails before `setColor(RED)`.